

Clustering and Anomaly Detection of Mixed Text Documents

Data Mining Assignment 3 Report

Student: Alperen Davran

Repository: https://github.com/alperendavran/assignment3_data_mining_clustering_anomaly_

May 2026

Abstract

This report studies a collection of 2,164 unlabeled text documents. The task has two goals. First, the documents must be grouped into meaningful article categories. Second, exactly 50 unsafe or corrupted files must be found as anomalies. I used one feature design for topic clustering and another feature design for anomaly detection. This was important because a topic cluster is about content, while an anomaly can also be about structure, repetition, URLs, and strange tokens. The final clustering output uses seven labels: six normal topic clusters and one anomaly cluster. The final anomaly output contains exactly 50 valid document IDs.

1 Problem and Main Idea

The assignment asks for a report, source code, a filled `clusters.csv`, and a filled `anomalies.csv`. It does not give true category labels, so the quality of the solution must be judged with internal scores and manual inspection.

The main idea is simple:

- For clustering, I use cleaned text, TF-IDF, LSA, and several clustering algorithms.
- For anomaly detection, I use structural features such as URLs, listing markers, coupon-like tokens, and repeated marketplace phrases.
- I do not choose the final model only by one score. I also inspect top terms, representative documents, cluster sizes, stability, and output format.

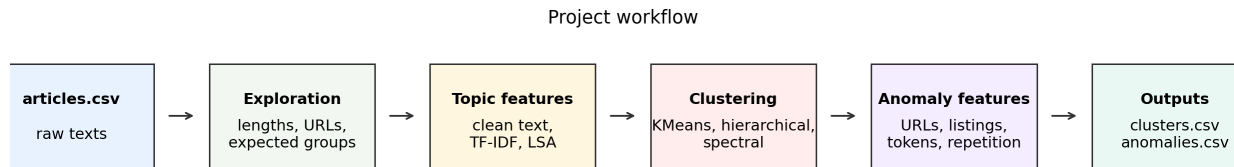


Figure 1: The project workflow. Topic clustering and anomaly detection use different feature sets because they answer different questions.

2 Data Exploration

The dataset contains 2,164 documents. The provided templates also contain 2,164 rows for cluster labels and 50 rows for anomaly IDs. The document IDs are unique, and the order of `clusters.csv` was checked against `articles.csv`.

Table 1: Dataset and output files.

Item	Value	Comment
Total documents	2,164	All documents have a <code>doc_id</code> and <code>text</code> .
Expected anomalies	50	The assignment states that 50 unsafe files were mixed in.
Final cluster labels	7	Six normal topic clusters plus one anomaly cluster.
Cluster limit	10	The final output stays below the assignment limit.

During exploration, clear topic signals appeared. I found many documents about space, hockey, computer graphics, cars, and medical topics. I also found a very strong unsafe-file pattern. Exactly 50 documents contained marketplace-like content such as listing IDs, pickup phrases, coupons, tokens, and suspicious links. This supported the decision to handle anomalies with structural features, not only with topic words.

3 Preprocessing

The minimum requirement is a bag-of-words representation. I included this baseline with `CountVectorizer`. Then I used TF-IDF as the main text representation.

The preprocessing steps were:

- remove English stopwords;
- remove very rare words with `min_df`;
- remove overly common words with `max_df`;
- remove URLs, e-mail addresses, long code-like tokens, and signature blocks for clustering;
- use TF-IDF with L2 normalization;
- use TruncatedSVD as LSA to reduce sparse text noise.

I did not use stemming as the main choice. Stemming can reduce the vocabulary, but it can also make top terms harder to read. In this assignment, clear explanation is important, so readable terms were more useful.

3.1 Why BoW and TF-IDF Have the Same Density

The bag-of-words matrix and the TF-IDF matrix have the same number of features and the same number of non-zero entries. This is normal. Both vectorizers used the same tokenizer, stopword list, `min_df`, and `max_df`. So they selected the same vocabulary and the same document-term positions. TF-IDF changed the values in these positions, not the positions themselves.

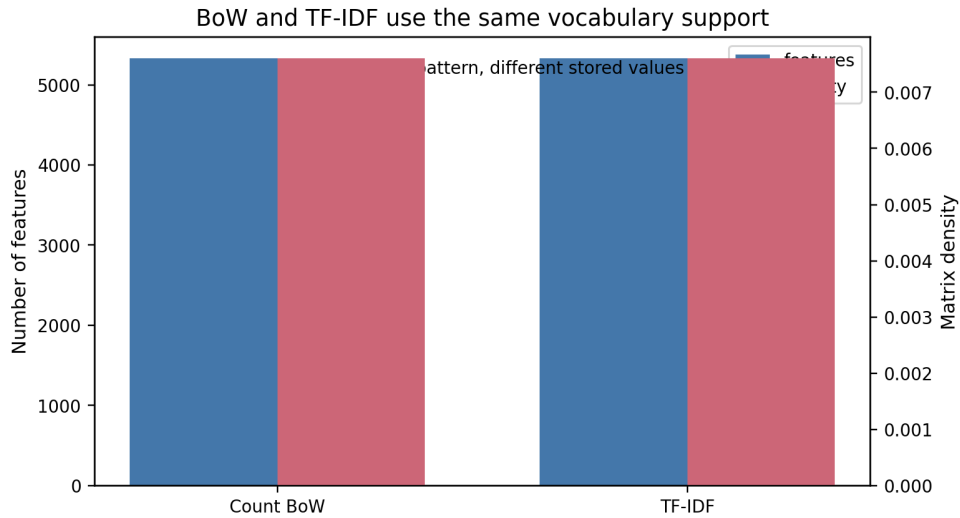


Figure 2: Count and TF-IDF use the same vocabulary support, but the stored values are different.

4 Clustering Methods

The assignment asks for at least two methods from k-means, hierarchical clustering, and spectral clustering. I ran all three:

- **KMeans:** strong baseline for normalized TF-IDF/LSA vectors;
- **Agglomerative clustering:** a hierarchical comparison method;
- **Spectral clustering:** a graph-based method tested as an extra check.

I used cosine silhouette for evaluation. Cosine similarity is a good choice for text because it compares the direction of vectors. This is better than comparing only raw size. I did not use Pearson correlation because Pearson mean-centers the vectors. In sparse text data, most entries are zero, and these many zeros can make Pearson hard to interpret. The absence of many words should not become a strong negative signal.

5 Clustering Results

Figure 3 shows the clustering scores. KMeans gave the best scores overall. KMeans with eight clusters had the highest silhouette, but it split some natural topics into smaller subtopics. KMeans with six normal clusters was easier to explain and more stable. Therefore I selected six normal topic clusters and kept the 50 unsafe files as a seventh cluster.

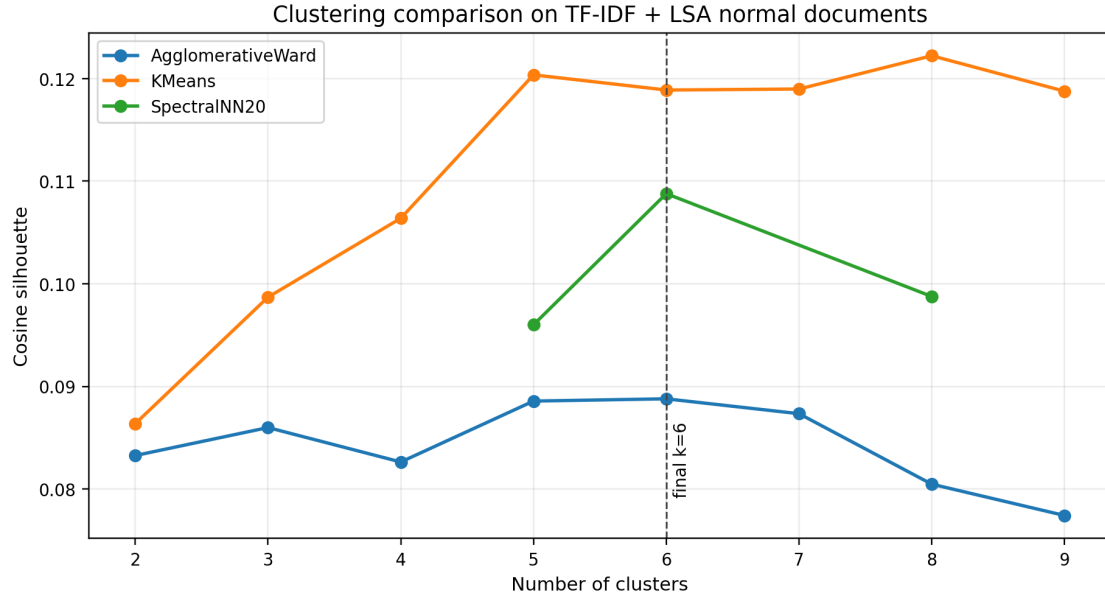


Figure 3: Clustering comparison on TF-IDF + LSA vectors. KMeans was the best method, but the final choice also used interpretability and stability.

Table 2: Main clustering comparison.

Method	Representation	k	Cosine silhouette	Comment
KMeans	TF-IDF + LSA	8	0.122	Highest score, but more split topics.
KMeans	TF-IDF + LSA	6	0.119	Final choice: clearer main categories.
Spectral	TF-IDF + LSA	6	0.109	Tested, but lower score and less balanced.
Agglomerative	TF-IDF + LSA	6	0.089	Useful comparison, but lower score.

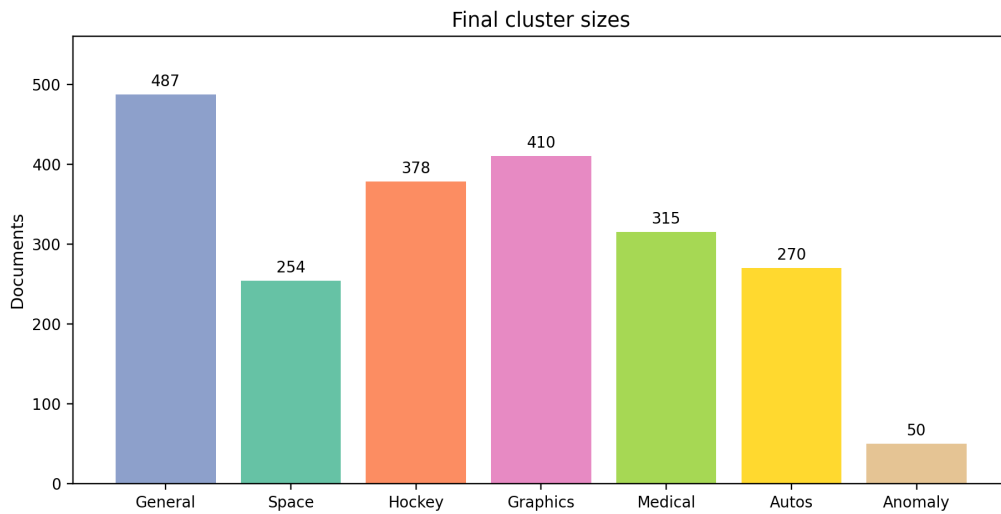


Figure 4: Final cluster sizes. The anomaly cluster contains exactly 50 files.

Table 3: Recovered clusters and top terms.

Label	Name	Size	Important terms
0	General discussion	487	just, like, people, know, question, time
1	Space	254	space, orbit, moon, launch, NASA, spacecraft
2	Hockey	378	game, team, hockey, players, NHL, playoffs
3	Graphics/software	410	graphics, program, files, image, Windows, software
4	Medical/health	315	disease, doctor, patients, pain, blood, treatment
5	Autos	270	car, cars, engine, dealer, Ford, drive
6	Unsafe listing/anomaly	50	listing, pickup, offer, coupon, token, discount

The clusters were not judged only by the score. I also inspected the top terms and representative documents for each cluster. This is important because text clustering is often difficult, and a small score difference can be less important than a clear and useful grouping.

6 Anomaly Detection

The anomaly task is different from topic clustering. A suspicious file may look different because of its structure, not only because of its vocabulary. For this reason, I built structural features:

- number of URLs;
- listing markers;
- coupon, token, SKU, and promo patterns;
- suspicious marketplace domains;
- repeated marketplace phrases;
- simple formatting ratios.

I compared Isolation Forest, k-nearest-neighbor distance, and Local Outlier Factor. Isolation Forest was selected because it agreed with the structural ranking and the kNN distance method on the same 50 files. LOF was less aligned with the final set.

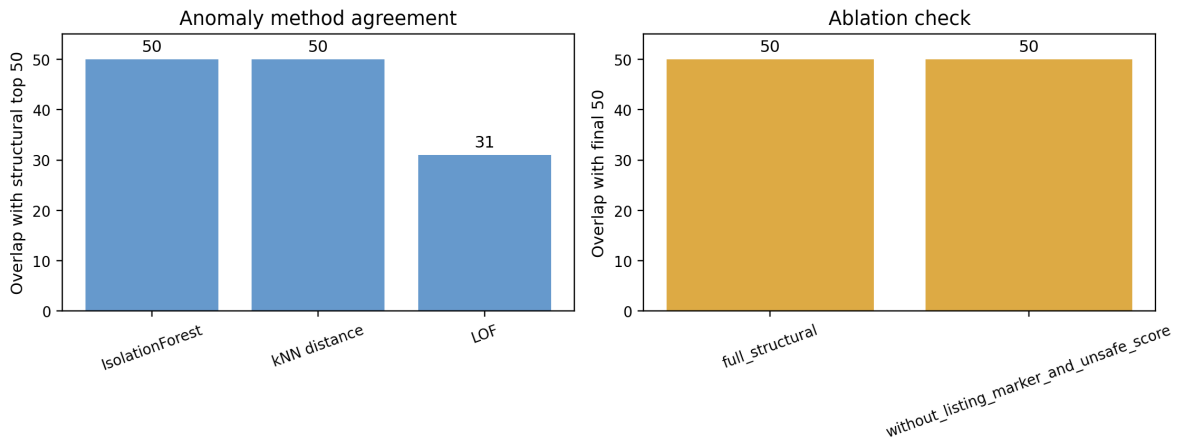


Figure 5: Anomaly validation. Isolation Forest and kNN agreed with the structural top 50. The ablation also found the same final 50 files.

Table 4: Anomaly method comparison.

Method	Selected	Overlap with structural top 50	Overlap with kNN top 50
Isolation Forest	50	50	50
kNN distance	50	50	50
LOF	50	31	31

I also ran an ablation test. In this test, I removed the exact `LISTING_ID` feature and the combined unsafe score. The model still found the same 50 documents. This is important because it shows that the result does not depend on only one regular expression.

7 Advanced Semantic Experiments

I also tested more advanced methods as a robustness check. These methods were not needed for the minimum assignment, but they help show that the final choice is not weak.

The advanced notebook used:

- Sentence-BERT embeddings with `all-MiniLM-L6-v2`;
- SBERT + KMeans;
- SBERT + agglomerative clustering;
- HDBSCAN;
- BERTopic;
- a UMAP visualization.

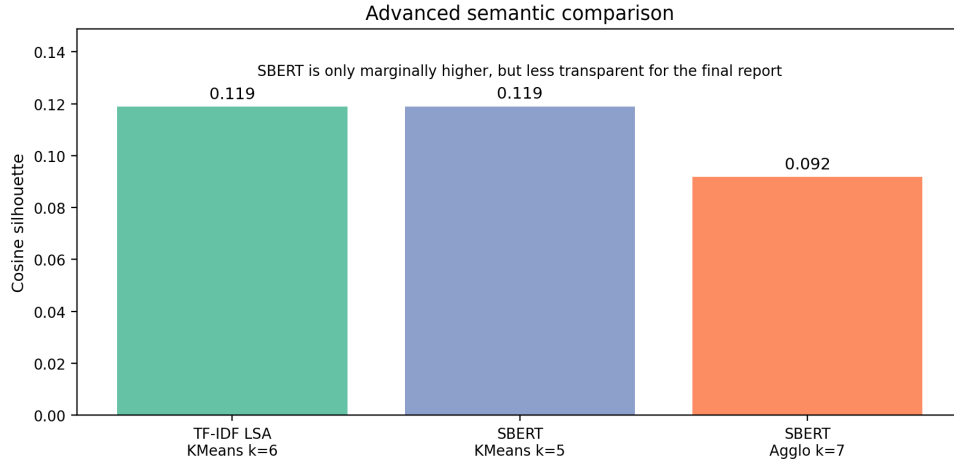


Figure 6: Advanced semantic comparison. SBERT KMeans was only slightly better by silhouette, but it was less transparent for the final report.

The best SBERT KMeans result was $k=5$ with silhouette 0.119. This was only a very small improvement over the final TF-IDF + LSA + KMeans $k=6$ result, which had silhouette 0.119 when rounded to three decimals. Also, the SBERT $k=6$ solution split the space topic into two parts. HDBSCAN marked more than 1,300 normal documents as noise, so it was not suitable for the required complete `clusters.csv`. BERTopic produced only three broad topics with the tested settings. For these reasons, I kept the final TF-IDF + LSA + KMeans solution.

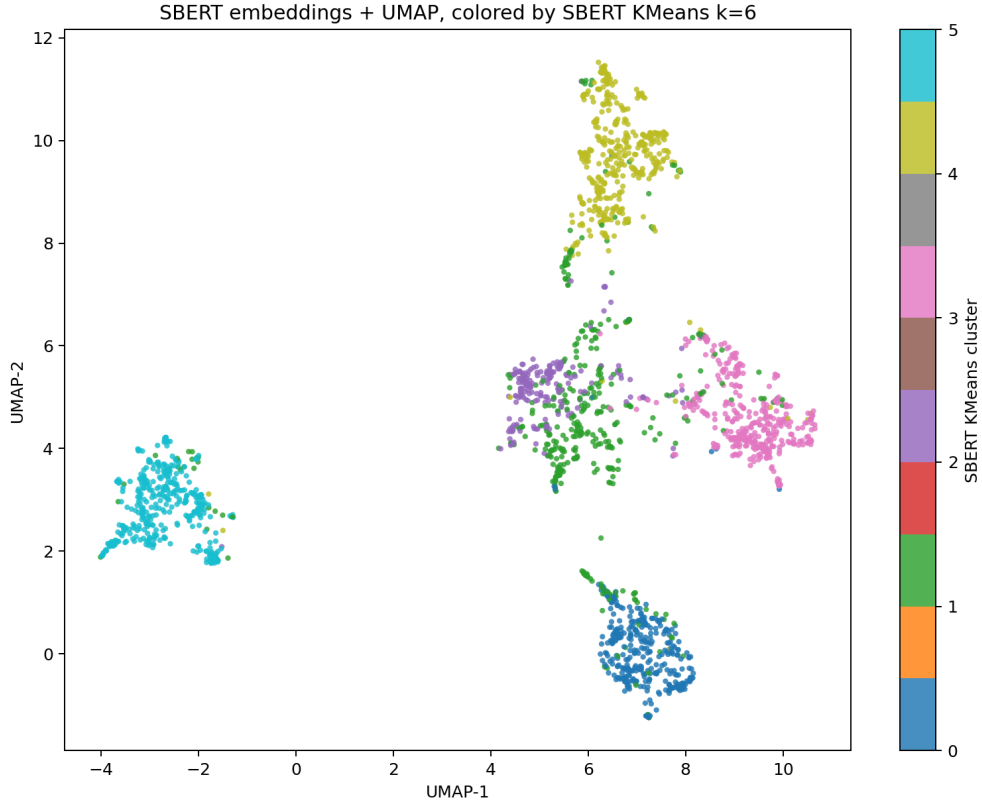


Figure 7: UMAP view of SBERT embeddings. This figure is used only as a visual check, not as the final metric.

8 Output Validation

The final files were checked by code.

Table 5: Final output validation.

Check	Result
<code>clusters.csv</code> has 2,164 rows	Passed
<code>clusters.csv</code> preserves article order	Passed
All cluster labels are integers	Passed
Number of final clusters is at most 10	Passed
<code>anomalies.csv</code> has exactly 50 IDs	Passed
Anomaly IDs are unique	Passed
Anomaly IDs exist in <code>articles.csv</code>	Passed

9 Limitations

There are no true category labels in the visible dataset. Because of this, I cannot report accuracy or F1 score for the clustering task. Silhouette is useful, but it is not enough by itself. This is why I also used top terms, example documents, cluster sizes, and stability checks.

Another limitation is that transformer embeddings are harder to explain. They are useful as an advanced comparison, but TF-IDF is easier to defend in this assignment because the top terms clearly show what each cluster means.

10 Conclusion

The final solution meets the assignment requirements. It uses several preprocessing options, compares KMeans, hierarchical clustering, and spectral clustering, stays below 10 clusters, and explains the recovered categories with important terms and representative documents. For anomaly detection, it compares Isolation Forest, kNN distance, and LOF, and it writes exactly 50 valid anomaly IDs.

The final model is not chosen only because of one score. It is chosen because it is simple, stable, interpretable, and compatible with the required output files. The advanced semantic experiments support this decision: they are interesting, but they do not clearly replace the final TF-IDF + LSA + KMeans pipeline.

References

- Scikit-learn. Text feature extraction documentation.
https://scikit-learn.org/stable/modules/feature_extraction.html
- Scikit-learn. Clustering text documents using k-means.
https://scikit-learn.org/stable/auto_examples/text/plot_document_clustering.html
- Scikit-learn. Isolation Forest documentation.
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- Scikit-learn. Local Outlier Factor documentation.
<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.LocalOutlierFactor.html>
- Reimers, N. and Gurevych, I. Sentence-BERT.
<https://arxiv.org/abs/1908.10084>
- BERTopic documentation.
<https://bertopic.readthedocs.io/>
- McInnes, L., Healy, J., and Melville, J. UMAP.
<https://arxiv.org/abs/1802.03426>
- Michael D. Ekstrand, CS 533 Assignment 6.
<https://cs533.ekstrandom.net/f22/assignments/a6/>

A Notebook and Output Files

The work is split into four notebooks:

- 01_data_exploration_preprocessing.ipynb
- 02_clustering_decision_log.ipynb
- 03_anomaly_detection_decision_log.ipynb
- 04_advanced_semantic_experiments.ipynb

The main output files are:

- clusters.csv
- anomalies.csv
- outputs/cluster_top_terms.csv
- outputs/cluster_representatives.csv

- `outputs/anomaly_detection_comparison.csv`
- `outputs/advanced_semantic_decision_summary.md`